

DOCKER & GITHUB ACTIONS

Complete Implementation Guide

For the Multi-Asset Trend Following Strategy | Architecture v3.2
Benefits · Installation · Development Workflow · Production Runtime

 Docker Reproducible runtime environment	 GitHub Actions Automated testing & deployment gates
---	---

Table of Contents

Part 1: Docker

1.1 What Problem Docker Solves

Without Docker, your pipeline runs in whatever Python environment happens to exist on the machine. This creates three categories of risk:

Environment drift	Your laptop has pandas 2.1; your VPS has pandas 1.5 — different resampling behaviour	Signals differ between machines
Dependency conflict	A system upgrade changes numpy's random seed behaviour	Backtest results change without any code change
Missing package	VPS is missing mplfinance; Script 15 crashes at 11 pm on rebalancing night	No charts; delayed decision

Python version	Local: Python 3.11. VPS: Python 3.9. F-string behaviour differs	Silent bug, hard to trace
API key exposure	Key is hardcoded in a script; accidentally committed to GitHub	API account compromised

Docker eliminates all of these by packaging your entire runtime — Python version, every library at a pinned version, your scripts, your config — into a single immutable image. Once built, that image runs identically on your laptop, a staging VPS, a cloud VM, or a colleague's machine.

1.2 Core Concepts

Image	A frozen, read-only snapshot of your entire runtime	A master CD you duplicate from
Container	A live running instance of an image	An installed copy of that CD, currently running
Dockerfile	The recipe that builds the image — step by step instructions	The manufacturing blueprint for the CD
Registry	A server that stores and distributes images (e.g. Docker Hub, GHCR)	A library where CDs are stored and checked out
Volume	A folder shared between your host machine and the container	A USB drive that both you and the container can read/write
docker-compose	A YAML file that defines how to run one or more containers together	A startup script that configures and launches everything

1.3 Installation

Windows

- Go to docs.docker.com/desktop/install/windows-install/
- Download Docker Desktop for Windows (requires WSL 2 — the installer guides you).
- Run the installer. Restart when prompted.
- Open Docker Desktop. Wait for the whale icon in the taskbar to show 'Docker Desktop is running'.
- Open a terminal and verify:

Verify Installation

```
docker --version
```

```
# Expected output: Docker version 26.x.x, build xxxxxxxx
```

macOS

- Go to docs.docker.com/desktop/install/mac-install/
- Download Docker Desktop for Mac (choose Apple Silicon or Intel based on your chip).
- Drag Docker.app to Applications.
- Open Docker Desktop. Accept the service agreement.
- Verify in terminal:

Verify Installation

```
docker --version
docker run hello-world # Should print 'Hello from Docker!'
```

Linux (Ubuntu/Debian)

Ubuntu / Debian

```
# Add Docker's official GPG key and repository
sudo apt-get update
sudo apt-get install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/
docker.asc

echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \
  https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo
$VERSION_CODENAME) stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-compose-plugin

# Allow running docker without sudo
sudo usermod -aG docker $USER
newgrp docker

# Verify
docker run hello-world
```

i Post-install note

On Linux, log out and back in after adding yourself to the docker group, or the group change won't take effect in your current shell session.

1.4 Your Dockerfile — Explained Line by Line

Create a file called *Dockerfile* in the root of your project (no file extension). This is the complete *Dockerfile* for your strategy:

✓ Why two stages?

The builder stage installs gcc and build tools needed to compile certain Python packages. Those tools are not needed at runtime. The two-stage build copies only the compiled packages into the final image, keeping it around 200 MB instead of 800 MB+.

1.5 docker-compose.yml — Your Control Panel

docker-compose lets you run your container without typing long docker run commands. It also defines the volumes, environment variables, and restart behaviour in one place.

docker/docker-compose.yml

```
# docker/docker-compose.yml
```

```
version: '3.9'
```

```
# — Shared configuration block (reused by all services) —————
```

```
x-strategy-base: &strategy-base
```

```
  image: trend-strategy:${STRATEGY_VERSION:-latest}
```

```
  volumes:
```

```
    # data cache persists between runs (your downloaded market data)
```

```
    - ../data_cache:/app/data_cache
```

```
    # logs persists between runs (run manifests, pipeline logs)
```

```
    - ../logs:/app/logs
```

```
  environment:
```

```
    # Keys come from your .env file — NEVER hardcode them here
```

```
    - EODHD_API_KEY=${EODHD_API_KEY}
```

```
    - YAHOO_FINANCE_KEY=${YAHOO_FINANCE_KEY}
```

```
  restart: 'no'    # Strategy runs on demand, not as a daemon
```

```
services:
```

```
# — Daily monitoring run —————
```

```
daily:
```

```
  <<: *strategy-base
```

```
  command: ["--mode", "daily"]
```

```
# — Monthly rebalancing run —————
```

```
monthly:
```

```
  <<: *strategy-base
```

```
  command:
```

```
    - "--mode"
```

```
    - "monthly"
```

```
    - "--account-equity"
```

```
    - "${ACCOUNT_EQUITY:-10000}"
```

```
# — Monthly dry-run (no files written) —————
```

```
monthly-dry:
```

```
  <<: *strategy-base
```

```
  command:
```

```
    - "--mode"
```

```
    - "monthly"
```

```
    - "--dry-run"
```

```
    - "--account-equity"
```

```
    - "${ACCOUNT_EQUITY:-10000}"
```

```
# — Backtest run —————
```

```
backtest:
```

<<: *strategy-base
command:
- "--mode"
- "custom"
- "--steps"
- "16,17,18,19,20"
— Interactive shell (debugging) —————
shell:
<<: *strategy-base
entrypoint: ["/bin/bash"]
stdin_open: true
tty: true

Create a .env file in the project root (add it to .gitignore immediately):

.env (project root — in .gitignore)
.env — NEVER commit this file
EODHD API KEY=your key here
YAHOO FINANCE KEY=your key here
ACCOUNT_EQUITY=12450
STRATEGY_VERSION=3.2.1

1.6 Development Workflow with Docker

Building the image

Run this from the project root whenever you change code or dependencies:

Build the image
Build the image and tag it with the current git SHA
GIT_SHA=\$(git rev-parse --short HEAD)
docker build \
--build-arg GIT_SHA=\${GIT_SHA} \
--build-arg STRATEGY_VERSION=3.2.1-dev \
--tag trend-strategy:dev \
--tag trend-strategy:\${GIT_SHA} \
.
Expected output ends with:
=> exporting to image
=> naming to docker.io/library/trend-strategy:dev

Running during development

Running containers
Quick daily check — see output in terminal
docker-compose -f docker/docker-compose.yml run --rm daily
Monthly rebalancing dry-run (no files written)

```

docker-compose -f docker/docker-compose.yml run --rm monthly-dry

# Open an interactive shell inside the container for debugging
docker-compose -f docker/docker-compose.yml run --rm shell

# Run a single script directly
docker run --rm \
  -v $(pwd)/data_cache:/app/data_cache \
  -v $(pwd)/logs:/app/logs \
  --env-file .env \
  trend-strategy:dev \
  --mode custom --steps 5,6,7

```

Iterating on code changes

During active development you don't need to rebuild the image for every file save. Mount your code as a volume to get live changes:

```

Live code mounting (dev only)
# Development override: mount local code into the container
# Changes to .py files take effect immediately – no rebuild needed
docker run --rm \
  -v $(pwd)/strategy:/app/strategy \
  -v $(pwd)/config:/app/config \
  -v $(pwd)/data_cache:/app/data_cache \
  -v $(pwd)/logs:/app/logs \
  --env-file .env \
  trend-strategy:dev \
  --mode custom --steps 5,6 --dry-run

# When you're done iterating, rebuild with the final code baked in:
docker build --tag trend-strategy:dev .

```

⚠ Important

The code-mounting trick is for development only. In staging and production, code must be baked into the image — never mounted. This ensures what you tested is exactly what runs.

Inspecting a running container

```

Useful inspection commands
# See all running containers
docker ps

# Tail logs from a running container
docker logs -f <container_id>

# Open a shell in an already-running container
docker exec -it <container_id> /bin/bash

# Inspect the image – see layers, build args, env vars
docker inspect trend-strategy:dev

# Check image size
docker images trend-strategy

```

1.7 Production Runtime with Docker

Building the release image

On release day (or in CI — see Part 2), build the production image with the official version tag:

```
Build and push release image
GIT_SHA=$(git rev-parse --short HEAD)
VERSION=3.2.1

docker build \
  --build-arg GIT_SHA=${GIT_SHA} \
  --build-arg STRATEGY_VERSION=${VERSION} \
  --tag ghcr.io/yourorg/trend-strategy:${VERSION} \
  --tag ghcr.io/yourorg/trend-strategy:latest \
  .

# Push to GitHub Container Registry
docker push ghcr.io/yourorg/trend-strategy:${VERSION}
docker push ghcr.io/yourorg/trend-strategy:latest
```

Running the monthly rebalancing in production

```
Production monthly run
# Pull the specific release image (never use :latest in production)
docker pull ghcr.io/yourorg/trend-strategy:3.2.1

# Run the monthly rebalancing
docker run --rm \
  -v /data/strategy/data_cache:/app/data_cache \
  -v /data/strategy/logs:/app/logs \
  --env-file /secrets/strategy.env \
  ghcr.io/yourorg/trend-strategy:3.2.1 \
  --mode monthly \
  --account-equity 12450 \
  --vix 18.4

# The PDF report appears in /data/strategy/logs/ on your host machine
```

Scheduling with cron

On your production Linux server, add these cron jobs:

```
/etc/cron.d/strategy
# crontab -e

# Daily monitoring: runs at 18:30 every weekday
30 18 * * 1-5 docker run --rm \
  -v /data/strategy/data_cache:/app/data_cache \
  -v /data/strategy/logs:/app/logs \
```

--env-file /secrets/strategy.env \
ghcr.io/yourorg/trend-strategy:3.2.1 \
--mode daily >> /data/strategy/logs/cron.log 2>&1
Monthly rebalancing: last trading day at 18:00
(Set the specific date each month — don't automate execution)
0 18 28-31 * * [\$(date +%d) -eq \$(cal awk 'NF{last=\$NF}END{print last}')] && \
docker run --rm \
-v /data/strategy/data_cache:/app/data_cache \
-v /data/strategy/logs:/app/logs \
--env-file /secrets/strategy.env \
ghcr.io/yourorg/trend-strategy:3.2.1 \
--mode monthly --account-equity 12450

Rollback in 60 seconds

Rollback procedure
A new version is misbehaving — roll back instantly
Step 1: Stop any running container
docker stop \$(docker ps -q --filter ancestor=ghcr.io/yourorg/trend-strategy:3.2.2)
Step 2: Re-run with the previous version
docker run --rm \
-v /data/strategy/data_cache:/app/data_cache \
-v /data/strategy/logs:/app/logs \
--env-file /secrets/strategy.env \
ghcr.io/yourorg/trend-strategy:3.2.1 \
--mode monthly --account-equity 12450
Previous image is still local — no pull needed — runs immediately

⚠ Keep 3 versions on disk

Always keep the last 3 release images pulled locally on your production server. docker image prune will delete unused images — make sure to exclude your last 3 versions before pruning.

Part 2: GitHub Actions

2.1 What Problem GitHub Actions Solves

Without automated gates, your change process relies entirely on human discipline — remembering to run tests, remembering to check backtest thresholds, remembering to validate config schema. For a trading strategy, forgotten checks translate directly into capital risk.

Parameter change goes live without backtest	PR is automatically blocked until backtest passes all 7 thresholds
---	--

Syntax error in a script discovered at midnight on rebalancing day	Caught within 2 minutes of the commit, before it reaches any server
No audit trail of what changed and when	Every run is logged in GitHub with who triggered it, what SHA ran, what passed/failed
Colleague deploys untested code to production	Branch protection + required status checks make this impossible
Docker image built inconsistently on different machines	Image built once in CI with identical inputs every time
Backtest PDF lives in someone's local folder	Attached to every GitHub Release permanently, accessible to whole team

2.2 Core Concepts

Workflow	A YAML file in <code>.github/workflows/</code> that defines when and what to run
Trigger (on:)	The event that starts a workflow — push, pull_request, schedule, manual button
Job	A group of steps that run on the same machine (runner) in sequence
Step	A single command or pre-built action inside a job
Runner	The virtual machine GitHub provides to execute your job (ubuntu-latest = fresh Ubuntu VM)
Action	A reusable step published by GitHub or the community (e.g. <code>actions/checkout@v4</code>)
Artifact	A file or folder saved from a workflow run — available for download for 90 days
Secret	An encrypted value stored in GitHub Settings — injected as an env var at runtime
Status check	A pass/fail result from a workflow job — can block PR merges if failed

2.3 Initial Setup

	Create the workflows directory In your project root, create the directory structure: <code>mkdir -p .github/workflows</code>
--	---

	Add your secrets to GitHub Go to your GitHub repository → Settings → Secrets and variables → Actions Click 'New repository secret' and add: EODHD_API_KEY — your EODHD API key YAHOO_FINANCE_KEY — your Yahoo Finance key GHCR_TOKEN — a GitHub Personal Access Token with packages:write scope TELEGRAM_BOT_TOKEN — your alert bot token (optional but recommended)
	Enable branch protection rules Go to Settings → Branches → Add rule Branch name pattern: main Enable: Require a pull request before merging Enable: Require status checks to pass before merging Add required checks: lint-and-test, config-validation Enable: Require branches to be up to date before merging Enable: Do not allow bypassing the above settings Repeat for: develop (same settings, but only 1 required review instead of 2)
	Add CODEOWNERS file Create .github/CODEOWNERS — this requires your review on any change to critical files: # Strategy config requires PM review config/strategy_params.yaml @your-github-username config/filter_thresholds.json @your-github-username # All strategy scripts require senior dev review strategy/ @your-github-username

2.4 Workflow 1: Lint and Test (runs on every push)

This is your fastest feedback loop. Every time anyone pushes any code, this runs in under 5 minutes and catches syntax errors, type errors, and failing unit tests before they spread.

<code>.github/workflows/lint_and_test.yml</code>
<code># .github/workflows/lint_and_test.yml</code>
<code>name: Lint and Test</code>
<code>on:</code>
<code> push: # Runs on every push to any branch</code>
<code> pull_request: # Also runs on every PR</code>
<code>jobs:</code>
<code> lint-and-test:</code>
<code> name: Lint, Type Check, Unit Tests</code>
<code> runs-on: ubuntu-latest</code>
<code> steps:</code>
<code> # Step 1: Check out your code onto the runner</code>
<code> - name: Checkout code</code>
<code> uses: actions/checkout@v4</code>
<code> # Step 2: Install Python 3.11 – exact version pinned</code>
<code> - name: Set up Python</code>
<code> uses: actions/setup-python@v5</code>
<code> with:</code>
<code> python-version: '3.11'</code>
<code> cache: 'pip' # Cache pip downloads between runs</code>
<code> # Step 3: Install all dependencies from the lockfile</code>
<code> - name: Install dependencies</code>
<code> run: </code>
<code> pip install poetry==1.8.2</code>
<code> poetry install --with dev</code>
<code> # Step 4: Ruff linting – catches unused imports, style issues</code>
<code> - name: Lint with Ruff</code>
<code> run: ruff check strategy/ config/</code>
<code> # Step 5: Black formatting check – fails if code is not formatted</code>
<code> - name: Check formatting with Black</code>
<code> run: black --check strategy/</code>
<code> # Step 6: Static type checking – catches type errors without running</code>
<code> - name: Type check with mypy</code>
<code> run: mypy strategy/ --ignore-missing-imports</code>
<code> # Step 7: Validate config schema – rejects malformed parameters</code>
<code> - name: Validate config schema</code>
<code> run: python ci/validate_config.py</code>
<code> # Step 8: Run all unit tests with coverage</code>
<code> - name: Run unit tests</code>
<code> run: pytest tests/unit/ -v --tb=short --cov=strategy --cov-report=xml</code>
<code> # Step 9: Upload coverage report as an artifact</code>
<code> - name: Upload coverage report</code>
<code> uses: actions/upload-artifact@v4</code>
<code> with:</code>
<code> name: coverage-report</code>
<code> path: coverage.xml</code>

i What you see in GitHub

After pushing, go to your repository → Actions tab. You'll see the workflow running in real time. Each step shows its output. If Step 4 (Ruff) fails, Steps 5–9 don't run — saving time. The PR shows a green ✓ or red ✗ next to each commit.

2.5 Workflow 2: Backtest Gate (runs on parameter-touching PRs)

This is the most important workflow for your strategy's integrity. It runs automatically whenever a PR touches strategy code or parameters, and blocks the merge if any of the 7 backtest thresholds fail.

```
.github/workflows/backtest_on_pr.yml
# .github/workflows/backtest_on_pr.yml
name: Backtest Regression Gate

on:
  pull_request:
    paths:
      # Only runs when these files change
      - 'config/**'
      - 'strategy/**'

jobs:
  backtest-gate:
    name: Backtest Regression
    runs-on: ubuntu-latest
    timeout-minutes: 60      # Kill if it runs over an hour

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'
          cache: 'pip'

      - name: Install dependencies
        run: |
          pip install poetry==1.8.2
          poetry install --only main

      # Restore cached fixture data so we don't need API keys in CI
      - name: Restore regression fixtures
        uses: actions/cache@v4
        with:
          path: tests/regression/fixtures/
          key: regression-fixtures-v1

      # Run the backtest engine on the out-of-sample window
      - name: Run backtest engine
        run: |
          python strategy/16_backtest_engine.py \
            --data-dir tests/regression/fixtures/ \
            --start-date 2018-01-01 \
```

--end-date	2023-12-31 \
--output	logs/backtest_results.json
# Check results against thresholds – exits with code 1 if any fail	
- name: Validate backtest thresholds	
run: python ci/check_backtest_thresholds.py logs/backtest_results.json	
# Run the look-ahead bias validator	
- name: Run look-ahead bias check	
run: python strategy/19_backtest_validator.py --results logs/backtest_results.json	
# Save the full report – visible in the PR even if thresholds pass	
- name: Upload backtest report	
if: always() # Upload even if thresholds failed (for diagnosis)	
uses: actions/upload-artifact@v4	
with:	
name: backtest-report-\${{ github.sha }}	
path: logs/	
retention-days: 90	

The threshold checker script that the workflow calls:

ci/check_backtest_thresholds.py
ci/check_backtest_thresholds.py
import json, sys
THRESHOLDS = {
'sharpe_ratio': ('>=', 0.80),
'max_drawdown': ('>=', -0.25), # drawdown is negative
'profit_factor': ('>=', 1.40),
'win_rate': ('>=', 0.40),
'stability_ratio': ('>=', 0.70),
'mc_5th_pct_sharpe': ('>=', 0.40),
'lookahead_bias_flag': ('==', 'PASS'),
}
results = json.load(open(sys.argv[1]))
failures = []
for metric, (op, threshold) in THRESHOLDS.items():
value = results.get(metric)
passed = eval(f'{value!r} {op} {threshold!r}')
status = 'PASS' if passed else 'FAIL'
print(f' {status} {metric}: {value} (required {op} {threshold})')
if not passed:
failures.append(metric)
if failures:
print(f'\n* {len(failures)} threshold(s) failed: {failures}')
sys.exit(1) # Non-zero exit blocks the PR merge
print('\n✓ All thresholds passed – PR cleared for review')

2.6 Workflow 3: Release Pipeline (runs on version tags)

When you push a version tag (e.g. `git tag v3.2.1` && `git push --tags`), this workflow builds the production Docker image, runs the full validation suite, and creates a GitHub Release with the backtest report attached.

<code>.github/workflows/release.yml</code>
<code># .github/workflows/release.yml</code>
<code>name: Release Pipeline</code>
<code>on:</code>
<code> push:</code>
<code> tags:</code>
<code> - 'v*.*.*' # Triggers on tags like v3.2.1</code>
<code>jobs:</code>
<code> release:</code>
<code> name: Build, Validate, and Release</code>
<code> runs-on: ubuntu-latest</code>
<code> timeout-minutes: 180 # Full suite can take up to 2.5 hours</code>
<code> permissions:</code>
<code> contents: write # Needed to create GitHub Release</code>
<code> packages: write # Needed to push Docker image to GHCR</code>
<code> steps:</code>
<code> - name: Checkout code</code>
<code> uses: actions/checkout@v4</code>
<code> # Extract version from tag (strips the 'v' prefix)</code>
<code> - name: Extract version</code>
<code> id: version</code>
<code> run: echo "VERSION=\${GITHUB_REF_NAME#v}" >> \$GITHUB_OUTPUT</code>
<code> # Verify the tag version matches strategy_params.yaml</code>
<code> - name: Verify version consistency</code>
<code> run: </code>
<code> PARAM_VERSION=\$(python -c \</code>
<code> "import yaml; d=yaml.safe_load(open('config/strategy_params.yaml')); \</code>
<code> print(d['meta']['strategy version'])")</code>
<code> if ["\$PARAM_VERSION" != "\${{ steps.version.outputs.VERSION }}"]; then</code>
<code> echo 'ERROR: Tag version does not match strategy_params.yaml'</code>
<code> exit 1</code>
<code> fi</code>
<code> # Run the complete backtest + Monte Carlo + validation suite</code>
<code> - name: Run full validation suite</code>
<code> run: </code>
<code> pip install poetry==1.8.2 && poetry install --only main</code>
<code> python strategy/16_backtest_engine.py --data-dir tests/regression/fixtures/</code>
<code> python strategy/18_monte_carlo_simulator.py --n-paths 10000</code>
<code> python strategy/19_backtest_validator.py</code>
<code> python strategy/20_oos_validator.py</code>
<code> python ci/check_backtest_thresholds.py logs/backtest_results.json</code>
<code> # Log in to GitHub Container Registry</code>
<code> - name: Log in to GHCR</code>
<code> uses: docker/login-action@v3</code>

with:
registry: ghcr.io
username: \${{ github.actor }}
password: \${{ secrets.GHCR_TOKEN }}
Build and push the production Docker image
- name: Build and push Docker image
uses: docker/build-push-action@v5
with:
context: .
push: true
tags:
ghcr.io/\${{ github.repository }}:\${{ steps.version.outputs.VERSION }}
ghcr.io/\${{ github.repository }}:latest
build-args:
GIT_SHA=\${{ github.sha }}
STRATEGY_VERSION=\${{ steps.version.outputs.VERSION }}
Create GitHub Release with validation report attached
- name: Create GitHub Release
uses: softprops/action-gh-release@v2
with:
generate_release_notes: true
files:
logs/backtest_results.json
logs/monte_carlo_report.pdf
logs/validation_report.pdf

2.7 Day-to-Day Usage

What you see when you open a Pull Request

lint-and-test / Lint with Ruff	Linting error in your code — click Details to see which file and line
lint-and-test / Type check with mypy	Type error — a function receives the wrong type somewhere
lint-and-test / Run unit tests	A unit test is failing — click Details to see which test and the assertion
lint-and-test / Validate config schema	strategy_params.yaml or filter_thresholds.json has an invalid value
backtest-gate / Backtest Regression	One or more of the 7 thresholds failed — download the backtest report artifact to see which metric
backtest-gate / Look-ahead bias check	The validator found future data leaking into a signal calculation

Making a parameter change — complete workflow

	Open an RFC issue Go to GitHub → Issues → New issue → choose the 'RFC: Parameter Change' template Describe: what you want to change, why, and what research supports it Tag it: [RFC] [PARAM]
	Create a feature branch git checkout develop git pull origin develop git checkout -b feature/rfc-42-increase-atr-multiplier
	Make the change and push Edit config/strategy_params.yaml — change the value Increment param_epoch by 1 Update effective_date to today git add config/strategy_params.yaml git commit -m 'param(risk): increase ATR initial stop 3.0 -> 3.2 (RFC #42)' git push origin feature/rfc-42-increase-atr-multiplier
	Open a PR to develop Go to GitHub → Pull Requests → New pull request Base: develop Compare: feature/rfc-42-increase-atr-multiplier The backtest gate starts automatically — watch it in the Actions tab While it runs, add the RFC issue number and a summary of expected impact to the PR description
	Review the gate results If green: download the backtest report artifact from the Actions run Share it with the Risk Manager for sign-off If red: click Details on the failing check to see which threshold failed Either revert the change or investigate why the metric regressed

	PR manager approves the PR on GitHub PM merges to develop At month-end: PR from develop → main, tag it v3.2.1-e3 Release workflow runs automatically, builds production image
--	--

Viewing workflow history

Go to your repository → Actions tab. You can filter by workflow name, branch, or status. Every run shows:

- Which commit triggered it and who pushed it
- How long each step took
- The full log output for each step
- Any artifacts uploaded (backtest reports, coverage)
- Whether status checks passed (determines if PRs were blocked)

Manually triggering a workflow

For the release workflow, you can also trigger it manually without pushing a tag:

Manual trigger via GitHub CLI
From GitHub UI: Actions → Release Pipeline → Run workflow
Or from the command line using the GitHub CLI:
gh workflow run release.yml \
--field version=3.2.1

Quick Reference

Docker — Most Used Commands

Docker command reference
Build image
docker build --tag trend-strategy:dev .
Run daily monitoring
docker-compose run --rm daily
Run monthly rebalancing (dry run)
docker-compose run --rm monthly-dry
Open shell for debugging
docker-compose run --rm shell
See what's running

<code>docker ps</code>
<code># Follow logs</code>
<code>docker logs -f <container_id></code>
<code># Pull latest production image</code>
<code>docker pull ghcr.io/yourorg/trend-strategy:3.2.1</code>
<code># Roll back to previous version</code>
<code>docker run ... ghcr.io/yourorg/trend-strategy:3.2.0 --mode monthly ...</code>
<code># Clean up old images (keep last 3 versions before running this)</code>
<code>docker image prune -a --filter 'until=720h'</code>

GitHub Actions — Most Used Commands

GitHub CLI reference
<code># Push code (triggers lint_and_test.yml automatically)</code>
<code>git push origin feature/my-branch</code>
<code># Open a PR (triggers backtest on pr.yml if config/ or strategy/ changed)</code>
<code>gh pr create --base develop --title 'param: increase ATR stop multiplier'</code>
<code># Create a release tag (triggers release.yml)</code>
<code>git tag -s v3.2.1 -m 'Release v3.2.1'</code>
<code>git push origin v3.2.1</code>
<code># Check status of latest workflow runs</code>
<code>gh run list --limit 10</code>
<code># Watch a running workflow live</code>
<code>gh run watch</code>
<code># Download artifacts from a run</code>
<code>gh run download <run-id> --name backtest-report</code>
<code># Re-run a failed workflow</code>
<code>gh run rerun <run-id> --failed-only</code>

Troubleshooting

docker: command not found	Docker not installed or not in PATH	Reinstall Docker Desktop; restart terminal
Permission denied connecting to Docker daemon	User not in docker group (Linux)	sudo usermod -aG docker \$USER then log out/in
Container exits immediately	Script error or missing env var	docker logs <id> to see the error; check .env file
Volume data not persisting	Wrong volume path or running as wrong user	Check -v paths; ensure host directory exists
Actions workflow not triggering	Branch name doesn't match the 'on: push: branches:' filter	Check the branches filter in the workflow file
Backtest gate not running on PR	Changed file is not in the 'paths:' filter	Add the file path to paths: in backtest_on_pr.yml

Secret not available in workflow	Secret not added to repo Settings or wrong name	Settings → Secrets → verify name matches \$ {{ secrets.NAME }}
Docker image too large (>500MB)	Dev dependencies included in final image	Verify --without dev in poetry export command